

Progress Report 2: Real-Time Cryptocurrency Streaming Pipeline

Student: Michael Hein (300375535)

Course: CSIS 4495 – Applied Research Project

Reporting Period: Feb 24 – Mar 1, 2026

1. Work Date/Hours Logs

Date	Hours	Description of Work Done
Feb 27	2.0	Researched AWS Athena and S3 Parquet integration patterns. Designed Hive-style partitioning strategy (<code>year=/month=/day=/hour=</code>) for raw trades and candles.
Feb 27	1.5	Implemented S3 Parquet sink (<code>s3_sink.py</code>)—consumes Kafka messages, buffers in memory, and flushes OHLCV candles to S3.
Feb 27	1.0	Updated centralized config and environment templates to include AWS S3, Athena, and sink tuning variables.
Feb 28	1.5	Implemented Athena setup script (<code>setup_athena.py</code>) with partition projection for automatic data discovery.
Feb 28	1.5	Built Athena query runner (<code>run_query.py</code>) to execute analytical queries (volume summaries, trade counts) on live S3 data.

Mar 1	1.0	Code cleanup: Deleted duplicate producers/consumers. Fixed import bugs and S3 prefix mismatches.
Mar 1	1.0	Documentation: Updated README.md with new architecture diagrams, tech stack tables, and AWS setup guides.
Mar 1	1.0	End-to-End Testing: Verified full pipeline flow (Coinbase → Kafka → S3 → Athena).

Total Hours This Period: 10.5

2. Summary Description of Work Done

This reporting period focused on establishing **AWS data persistence** and performing a comprehensive code quality audit. The primary milestone was the successful deployment of the **S3 + Athena integration**, enabling historical analysis of streaming trade data that was previously volatile.

Technical Achievements:

- **Storage Strategy:** Implemented a Kafka-to-S3 sink that flushes raw trades and 1-minute OHLCV candles as Parquet files. I used **Hive-style partitioning** to ensure query efficiency and minimize costs by limiting data scans.
- **Automated Discovery:** Configured **Athena Partition Projection**. This allows Athena to calculate partition locations based on the S3 folder structure dynamically, removing the need for manual [MSCK REPAIR TABLE](#) commands as new data arrives every hour.
- **Infrastructure Fixes:** Resolved a critical **AWS region mismatch** ([us-west-2](#) vs [ca-west-1](#)) that caused silent failures during initial testing.

Project Health & Cleanup:

I performed a "spring cleaning" of the repository, removing three redundant files that were causing versioning confusion. I also corrected the Athena hourly volume query, which was previously aggregating data across multiple days incorrectly; it now accurately reflects specific hourly windows.

3. Repo Check-In: Implementation Details

New Files (S3 + Athena Integration)

- [Implementation/athena/s3_sink.py](#) — The core logic for buffering Kafka streams into S3 Parquet.
- [Implementation/athena/setup_athena.py](#) — DDL scripts for database and table creation.
- [Implementation/athena/run_query.py](#) — A utility for developers to run CLI-based analytical queries.

Modified/Deleted Files

- **Modified:** [README.md](#), [config/env.example](#), [.env](#), and [setup_aws.sh](#).
- **Deleted:** Redundant copies of [coinbase_producer.py](#), [spark_stream.py](#), and [streamlit_app.py](#) to ensure a single source of truth.

4. AI Use Section

AI Tool Name	Feature for Which the AI Tool Was Used	Value Addition
Cursor (Claude)	Code Review & Cleanup	Identified duplicate files and import bugs that were overlooked during rapid development.
Cursor (Claude)	Partition Projection Design	Provided specific DDL syntax for Athena partition projection, saving hours of documentation research.
Cursor (Claude)	Troubleshooting	Diagnosed the InvalidRequestException as a region mismatch between the bucket and the client.

5. Appendix: AI Prompt History

The following prompts were used to guide the development of the AWS integration and cleanup phases:

- **Prompt 1 — S3 sink design:** "I need to write a Kafka consumer that reads from `crypto.trades.raw`, buffers messages, and periodically flushes them as Parquet files to S3 with Hive-style partitioning (`year/month/day/hour`). It should also compute 1-minute OHLCV candles and write those as a separate Parquet dataset. Can you help me structure this?"

- **Prompt 2 — Athena table setup:** "How do I create Athena external tables over my S3 Parquet data? I want partition projection so I don't have to run MSCK REPAIR TABLE every time new data arrives. The partitions are year, month, day, hour as strings."
- **Prompt 3 — Athena query examples:** "Write me example Athena queries for my crypto pipeline: recent trades, trade count by symbol, latest 1-minute candles, and an hourly volume summary. The tables are raw_trades and candles_1m in the crypto_pipeline database."
- **Prompt 4 — Code review and cleanup:** "How is the project going? Can you review the codebase and identify any issues, duplicates, or things that need fixing?"
- **Prompt 5 — Bug fixes:** "Can you fix all the issues you found — remove the duplicate files, fix the import bug, fix the S3 prefix mismatch, fix the Athena query, and update the README?"
- **Prompt 6 — AWS configuration:** "How do I configure AWS credentials for the Redshift and Athena sinks?"
- **Prompt 7 — Athena setup error:** "I'm getting an InvalidRequestException when running setup_athena.py — it says the S3 location is invalid. Here's the error output."
- **Prompt 8 — End-to-end testing:** "How do I test the full pipeline? I want to run the producer, S3 sink, and Athena queries together."
- **Prompt 9 — README update:** "Update the README to include documentation for the Redshift and Athena sinks, the new AWS environment variables, and setup instructions for both sink options."